Rewriting multi-platform apps with Claude Code

One person. Three days. Seven platforms.

Don Browning

What This App/Service Is

Volusia County operates ~30 vehicle beach access ramps that open and close throughout the day based on tides, weather, capacity, and turtle nesting. Claude and I built an app that polls the county GIS system every 60 seconds and serves that data alongside NOAA tide predictions, water temperature, and NWS weather — so you can check conditions before you load up the truck.

3 Days
Part-Time

From blank repo to fully deployed multi-platform application

5 languages • 7 platforms • 1 monorepo

Overview:

- **Go backend** API server, data ingester, PostgreSQL migrations, external web service integration
- **Swift apps** iOS, iPadOS, watchOS, tvOS — all sharing a single Swift package
- **Responsive website** Vanilla HTML/JS + Tailwind CSS with tide charts, dark mode, live webcam
- **IoT templates** TRMNL e-ink display + backward-compatible Tidbyt device
- **Infrastructure** Dockerfile, GitHub Actions CI, DigitalOcean App Platform deploy

System Architecture

Single Go binary handles data ingestion + API serving — clients consume via REST

DATA SOURCES

Volusia County GIS
ArcGIS MapServer — poll every 60s

NOAA Tides & Currents
Station 8721164 — predictions + water temp

National Weather Service
NWS API — forecast + conditions

Go Service (Echo v4)

API Server | Data Ingester | NOAA Client | Weather Client

internal/handlers | internal/ingester | internal/noaa | internal/weather

PostgreSQL
DO Managed DB

ramp_status | ramp_status_history

REST ENDPOINTS

/api/v2/ramps | /api/v2/tides | /api/v2/weather | /api/v2/trmnl | /rampstatus (v1) | /tides (v1)

CLIENT PLATFORMS

Web App
HTML/JS + Tailwind

iOS App
SwiftUI

watchOS
SwiftUI

tvOS
SwiftUI

TRMNL
Liquid template

Tidbyt
Pixlet (frozen)

All Platforms Shippable

Six implementation phases complete — every platform reached a shippable baseline

Go API	PRODUCTION	v1 + v2 endpoints, GIS ingester, NOAA tides/temp, NWS weather — running on DigitalOcean App Platform	`Go · Echo v4 · pgx · slog`
Website	PRODUCTION	Responsive dashboard with ramp grid, tide chart, weather, webcam, dark mode, favorites, PWA	`HTML · JS · Tailwind CSS`
iOS + iPad	BUILDABLE	Universal SwiftUI app with ramps, tide chart, weather, webcam, city/status filtering	`SwiftUI · async/await · SPM`
watchOS	BUILDABLE	Glance-first ramp status with NSB default and all-cities drill-down	`SwiftUI · WatchKit`
tvOS	BUILDABLE	Ambient dashboard with ramps, tide chart, weather, auto-refresh, Siri Remote navigation	`SwiftUI · Focus engine`
TRMNL	READY	Monochrome e-ink template: 4 NSB ramps, tide bar, water temp, local clock	`Liquid · HTML`

The Build Timeline

6 phases across 3 days — each phase built on the last

DAY 1 — MAR 9

1 Foundation

Go API + ingester, PostgreSQL schema, v1 compat endpoints, NOAA client

`Custom migration runner with go:embed — no external library`

DAY 2 — MAR 10

2 Website

Tailwind site, canvas tide chart, NWS weather, PWA, dark mode, favorites

`~500 lines vanilla JS, zero frameworks, no build step`

3 CI/CD + Deploy

GitHub Actions, DigitalOcean App Platform, live at donwb.com

`lint → test → build → deploy + smoke test`

DAY 3 — MAR 11

4 iOS App

Universal SwiftUI (iPhone + iPad), shared BeachStatus Swift package

`Shared SPM package for cross-platform reuse`

5 Watch + TV

watchOS glance app, tvOS ambient dashboard, programmatic app icons

`Separate theme files per platform — avoids cross-compilation`

6 TRMNL

E-ink Liquid template, monochrome status differentiation

`Pure CSS, no JS — reused v2 endpoints, zero backend changes`

Engineer-Driven, AI-Built

No code editor. All work done in Claude Code — but every architectural decision was human-made.

The Workflow

1. **Write detailed requirements first**

   1,100-line REQUIREMENTS.md — the spec drove everything. Claude Code consumed it phase by phase.

2. **Make architecture decisions**

   Human chose: Echo v4 over Chi, custom migration runner over golang-migrate, goroutine ingester over separate process.

3. **Claude Code implements**

   All code written in the terminal. No VS Code, no Xcode (except project scaffolding). Every Swift, Go, HTML, and Liquid file.

4. **Document decisions back into the spec**

   Each phase ends with "Key decisions made" — the requirements doc is both input and living record.

Architecture Calls (Human-Made)

`Single binary: API + ingester in one process`
Goroutine-based ingester avoids a separate worker — simpler deploy, one service on App Platform

`1,100-line requirements doc before any code`
Phased spec drove the entire build — Claude Code consumed it phase by phase as the source of truth

`Shared Swift Package across 3 Apple targets`
Models, networking, utilities written once — iOS, watchOS, tvOS all depend on one SPM package

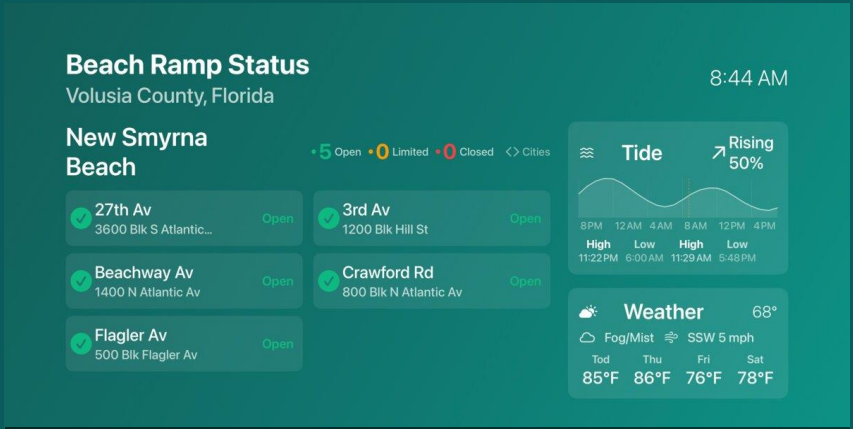
`Custom migration runner, no external library`
go:embed bakes SQL files into the binary — zero runtime dependencies, auto-runs at startup

`External AWS RDS instead of DO managed DB`
Same Postgres instance for dev and prod — simpler than managing two databases

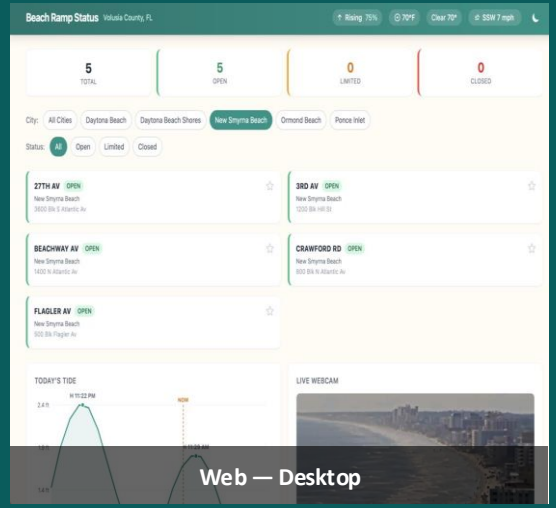
`Canvas API tide chart, no charting library`
~500 lines vanilla JS total on the website — zero frameworks, zero build step

One Codebase, Every Screen

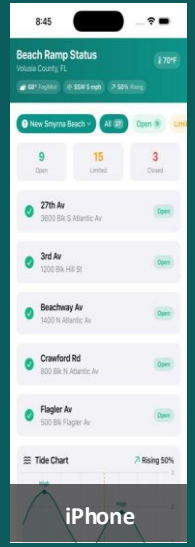
Live on 7 platforms — built part-time over 3 days with Claude Code



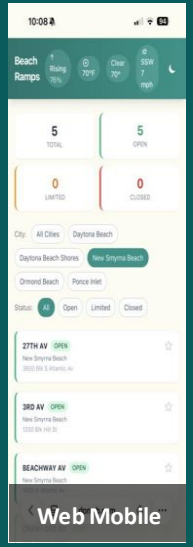
Apple TV



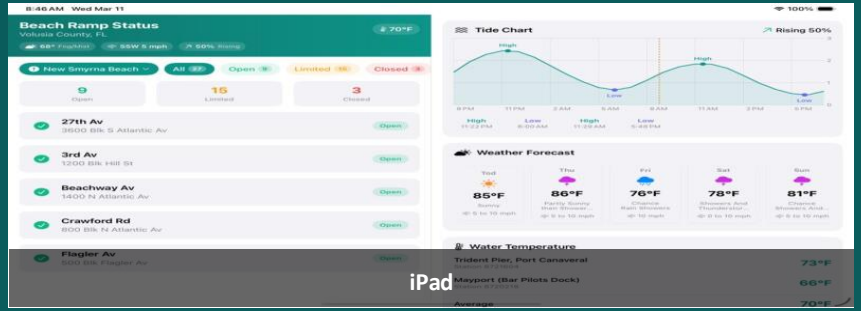
Web — Desktop



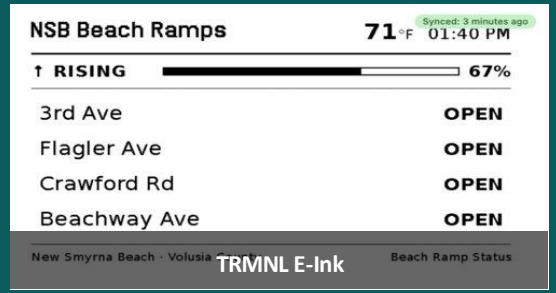
iPhone



Web Mobile



iPad



TRMNL E-Ink

Apple Watch